CLOUD COMPUTING AND BIG DATA

Assignment 2 (CO2)

Analyse, Evaluate and Validate a Virtualization Architecture for a Multi-Tenant University Private Cloud (EduCloud)

Name: SHANE MAC S Roll No.: 192211511

Course / Section: CSA1505

Common assumptions used throughout

The assignment fixes the host parameters. Everything else is an explicit, justified assumption so that every formula in Task 3 can be recomputed and later calibrated against the experiments.

Symbol	Value	Justification
P	64 schedulable pCPUs per host	Treated as hardware threads; one SMT sibling pair gives gamma = 1.25x one thread (commonly reported 20-30% SMT gain). Disabling SMT leaves 64/1.25 = 51.2 pCPU-equivalents.
M, M_h	512 GB; host reserve 16 GB (KVM), 12 GB (containers, gVisor)	Host kernel, page cache, vhost threads, hypervisor structures. KVM reserves more for QEMU/vhost/EPT.
v, m	v = 4 vCPUs, m = 8 GB per instance	m is given; 400 VMs / 16 hosts = 25 VMs per host, so 25 x 4 = 100 vCPUs per host, an average 1.56:1 overcommit.
u	0.2 normal, 0.6 registration peak	Peak is stated as about 3x normal; 3 x 0.2 = 0.6 average vCPU utilisation.
t_exit, t_sys	1.5 us, 0.2 us	Given in the assignment.
f0, alpha	2000 exits/s per vCPU; alpha = 0.25	Timer, IPI, HLT and EOI exits with APICv; extra preempt/resume exits grow 25% for each extra vCPU per pCPU.
f_io, b	20,000 virtio events/s per vCPU, batch b = 4	Web+DB mixed load; event-idx/notification suppression batches 4 events per kick.
Lambda, dW	2x10^5 TLB misses/s/vCPU, extra nested-walk cost 60 ns	Nested (2-D) walk needs up to 24 memory references versus 4 native; most are cached, ~60 ns extra per miss.
f_sys, delta_sec	2x10^5 syscalls/s/vCPU, +0.05 us per syscall (seccomp-BPF)	Typical web/DB mix; seccomp filter adds about 25% to a 0.2 us syscall. Plus 0.5% cgroup accounting.
t_gv	2.0 us per intercepted syscall	Derived from the given numbers: one trap into the user-space kernel costs about one VM-exit (1.5 us) plus 0.5 us Sentry processing.
B, eta	25 Gbps; goodput efficiency 0.941	1448 payload bytes per 1538 wire bytes (preamble, headers, inter-frame gap). Second NIC carries storage/migration traffic.
c_pkt	0.55 us (KVM/vhost), 0.75 us (veth+bridge+iptables), 3.0 us (gVisor netstack)	Per 1500-byte packet CPU cost; native kernel with offloads about 0.35 us. To be calibrated by iperf3 (Task 8).
k_net, kappa	4 cores for packet processing; kappa = 0.02 / 0.03 / 0.05	Contention on the virtual switch grows with log of the number of ports: efficiency g(n) = 1/(1 + kappa ln n).
NVMe	500k 4 KiB IOPS, 3 GB/s per host; per-I/O CPU cost 4.4 / 3.3 / 19 us (KVM / runc / gVisor); 8 cores for I/O	KVM adds virtio copy plus t_exit/b; runc adds overlayfs; gVisor adds about 3 intercepted syscalls (3 x 2.0 us) plus a Gofer RPC (about 10 us).

Symbol	Value	Justification
KSM, balloon	15% page sharing, 20% balloon reclaim	Same-OS guests share kernel and library pages; idle guest memory reclaimable.

Task 1: Characterisation of the Virtualization Architectures

Three architecturally different approaches are selected: **A** a hardware-assisted, fully virtualized VM (KVM/QEMU with paravirtual virtio I/O) as the VM-based approach; **B** OS-level containers (Docker/runc) as the container-based approach; **C** a sandboxed container runtime with a user-space kernel (gVisor/runsc). They differ in who enforces isolation: the CPU hardware (A), the shared host kernel (B), and a user-space kernel plus seccomp (C). They are not two configurations of one hypervisor.

Why the others were not selected. A Type-2 hosted hypervisor (VirtualBox on a desktop OS) adds a second scheduler and a general-purpose host OS under the VMs, which is unsuitable in a data centre; KVM already is a Type-1-style hypervisor because the hypervisor runs in the host kernel with direct hardware access. Pure paravirtualization (Xen PV) is obsolete on VT-x/EPT hardware; its useful part (paravirtual drivers) is already inside Approach A as virtio. Kata/Firecracker micro-VMs run on KVM, so they would be a configuration of Approach A and not counted as a separate approach; they are reconsidered in Task 12.

1.1 Approach A: KVM/QEMU, hardware-assisted full virtualization

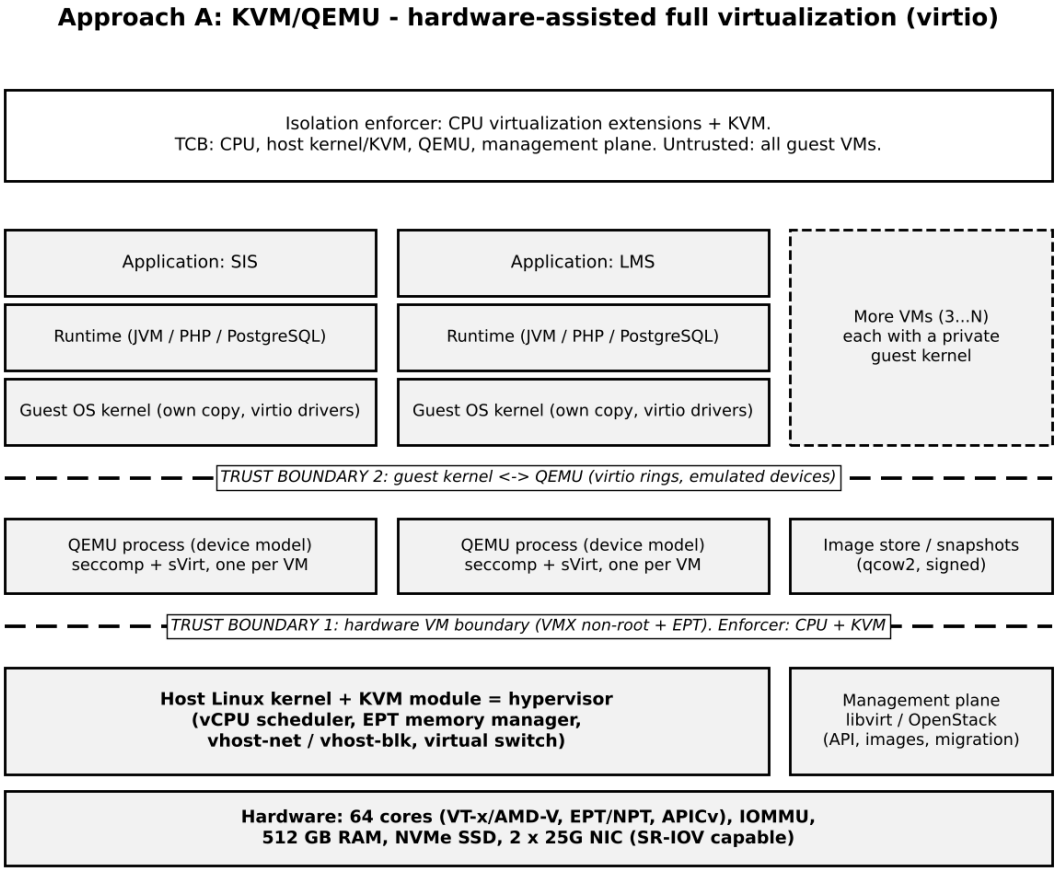


Figure 1: Layered architecture and trust boundaries of Approach A (KVM/QEMU).

1.2 Approach B: Docker/runc containers

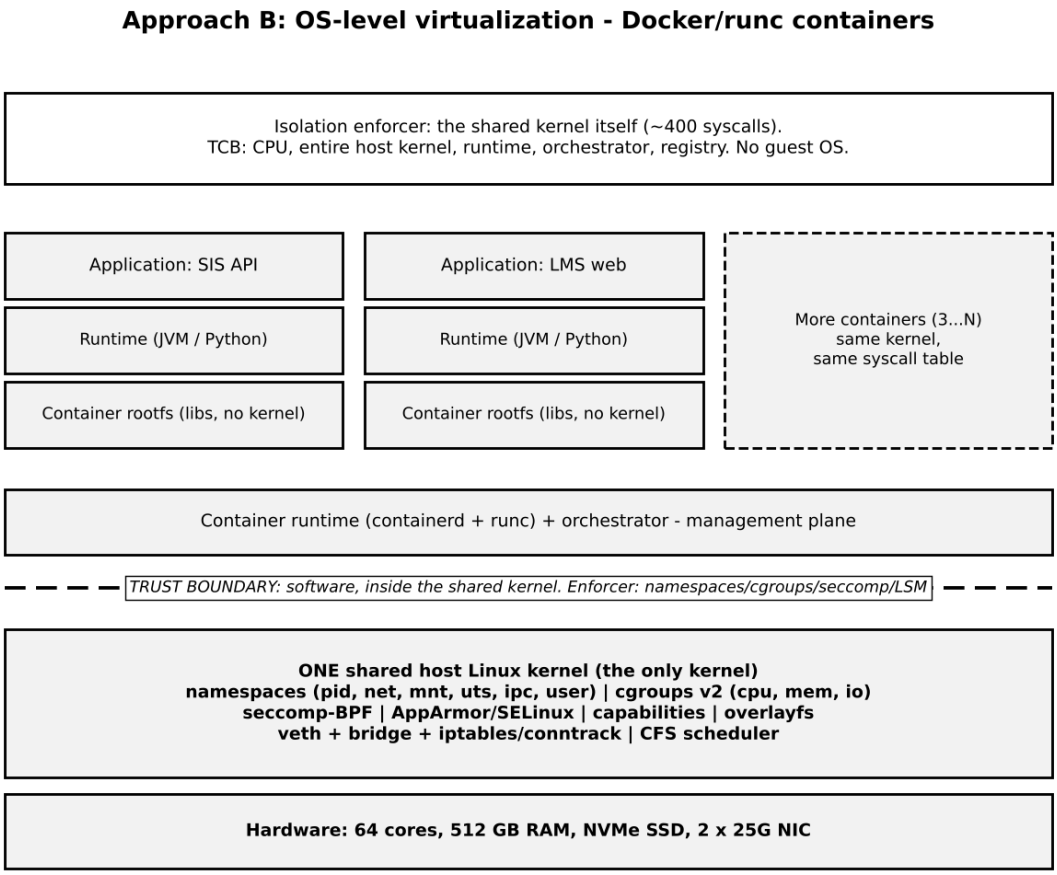


Figure 2: Layered architecture and trust boundaries of Approach B (runc containers).

1.3 Approach C: gVisor sandboxed container runtime

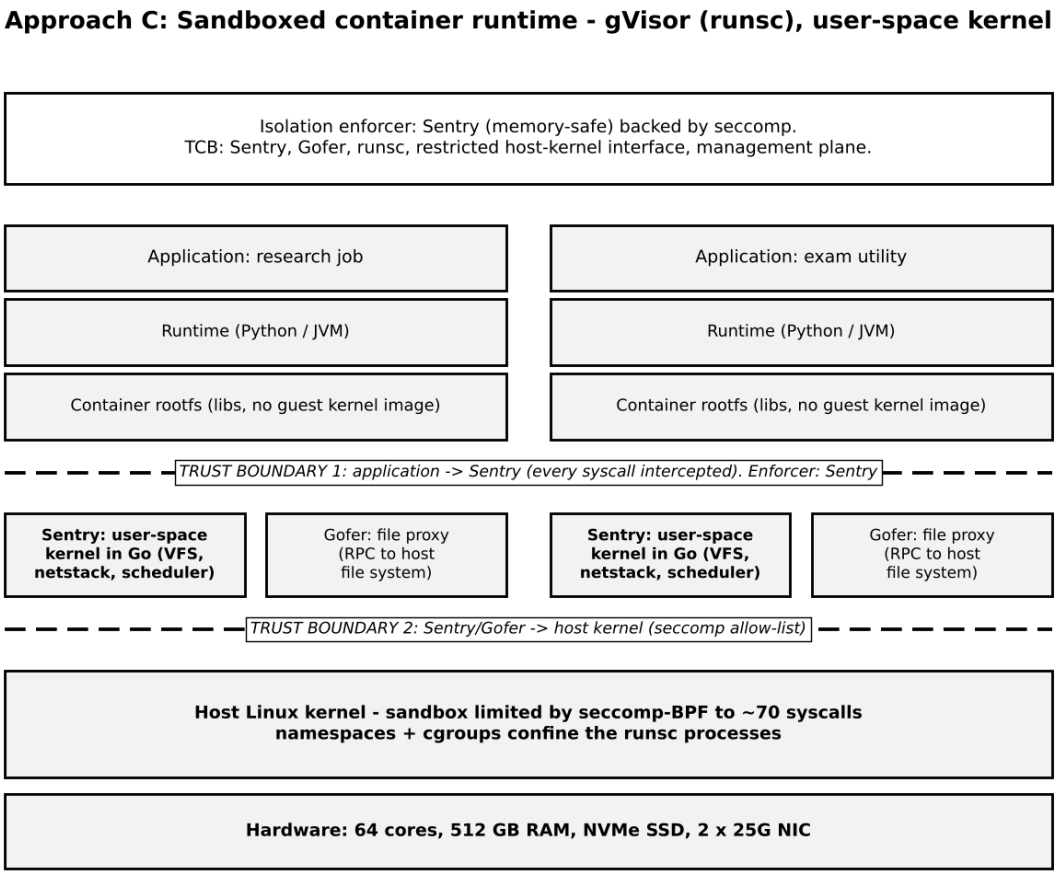


Figure 3: Layered architecture and trust boundaries of Approach C (gVisor).

1.4 Isolation, resource virtualization and relevance

Aspect	A: KVM/QEMU VM	B: runc container	C: gVisor sandbox
Isolation enforcer	CPU virtualization extensions (VMX non-root mode, EPT) and the KVM module. Guest cannot touch host memory.	The single shared host kernel: namespaces, cgroups, seccomp, LSM.	Sentry (user-space kernel written in Go) backed by a seccomp allow-list of about 70 host syscalls.
Trust boundaries	(1) Guest to hypervisor (hardware, VM exit). (2) Guest kernel to QEMU device model (virtio). (3) Management plane (libvirt/OpenStack) to hosts.	(1) Container to host kernel through about 400 syscalls. (2) Runtime/orchestrator to host (root-level). No separate kernel per tenant.	(1) App to Sentry (every syscall intercepted). (2) Sentry/Gofer to host kernel (seccomp). (3) Management plane.
CPU	vCPU = host thread; hardware runs guest code natively; privileged events cause VM exits; the host scheduler time-slices vCPUs on pCPUs, so overcommit produces steal time.	Processes scheduled directly by the host CFS scheduler; cgroup cpu.weight / cpu.max enforce shares and caps. No traps.	Same host scheduling of the sandbox threads; additionally each app syscall traps into the Sentry (ptrace, KVM or systrap platform).
Memory	Guest physical to host physical through second-level EPT page tables; ballooning, KSM, host swap available for overcommit.	Ordinary virtual memory; cgroup memory.max limit; exceeding it triggers reclaim then OOM-kill (no balloon, no per-tenant swap by default).	Sentry manages the app memory map and backs it with a host memfd; extra memory for Sentry and Gofer; page cache can be duplicated.

Aspect	A: KVM/QEMU VM	B: runc container	C: gVisor sandbox
Storage	virtio-blk/virtio-scsi to QEMU or vhost; qcow2/raw image on shared backend or local NVMe; SR-IOV/NVMe passthrough possible.	overlayfs layers and bind-mounted volumes; direct host file-system path, near-native.	File operations go app -> Sentry VFS -> Gofer RPC -> host file system; strong interposition, high per-I/O cost.
Network	virtio-net with vhost-net into a virtual switch (Linux bridge / Open vSwitch / SDN overlay); SR-IOV VF possible.	veth pair per container into host bridge, iptables/conntrack NAT, overlay networks (VXLAN) for multi-host.	Own user-space TCP/IP stack (netstack) in the Sentry; packets exchanged with host through a veth/AF_XDP style interface.
Relevance to EduCloud	Strong tenant boundary between departments; each VM has its own patchable kernel; live migration supports host patching. Natural fit for SIS database and exam portal.	Highest density and fastest start-up; suitable for stateless LMS web tiers and trusted internal batch jobs; weakest boundary (shared kernel).	Smaller kernel attack surface than runc without a full guest OS; suitable for running untrusted code (student programming submissions, exam sandboxes) with low I/O.

Task 2: Dominant Overhead Source for Each Approach

For each approach the dominant overhead is the mechanism that sits on the most frequently executed path of the architecture and whose cost grows as co-location or I/O rate grows. The model in Task 3 uses exactly these mechanisms.

A: KVM - VM exits caused by virtual I/O notifications and interrupt handling

- **What.** A VM exit saves guest state, switches to host mode, handles the event (timer write, IPI, HLT, virtio queue kick, EPT violation) and re-enters. Each costs t_exit = 1.5 us, which is about 7.5x one native syscall.
- **Where.** At the hardware boundary between guest and KVM (trust boundary 1 in Figure 1) and, for I/O, continuing into vhost/QEMU (boundary 2). CPU-bound guest code does not exit; the exit rate is set by interrupts and I/O.
- **Why it is the right focus.** Exits are the only extra work on the normal execution path for a hardware-assisted VM; nested page-table walks are the second term but are mitigated by huge pages. For the registration-week mix (many small network and disk requests) exit rate is large.
- **Scaling.** Exits per second per vCPU grow with I/O rate (f_io / b) and with overcommit, because every preempt/resume of a vCPU adds timer, IPI and HLT exits: f_exit(r) = f0(1 + alpha(r-1)) + f_io/b. At r = 8 the base rate is 2.75x that at r = 1. Larger batch b or polling (vhost busy-poll, posted interrupts) lowers the cost.

B: runc containers - virtual switch and netfilter processing (veth, bridge, conntrack) plus seccomp on syscalls

- **What.** There is no hypervisor, so CPU and memory paths are native. The extra cost is per-packet processing through veth, the Linux bridge, iptables and the conntrack table, plus a seccomp-BPF filter run on each syscall.

- **Where.** Inside the shared host kernel networking stack (softirq context) and at syscall entry. These costs are charged to the host, not to any tenant, which also makes them a shared resource.
- **Why.** Registration traffic (logins, HTTP) is packet-heavy, so per-packet cost, not CPU virtualization, governs scalability for containers. seccomp adds only about 1% CPU in our model.
- **Scaling.** Per-packet cost is nearly constant (0.75 us), but total packet rate and conntrack/bridge lock contention grow with the number of containers sharing the bridge; modelled as g(n) = 1/(1 + 0.03 ln n). Throughput stays flat until softirq cores saturate.

C: gVisor - system-call interception by the Sentry

- **What.** Every application syscall is trapped and re-implemented in a user-space kernel. Cost is t_gv = 2.0 us instead of 0.2 us, i.e. 1.8 us extra per syscall (9x).
- **Where.** Trust boundary 1 in Figure 3 (application to Sentry). File I/O adds a second hop to the Gofer (RPC), and network packets go through the Go netstack.
- **Why.** The isolation guarantee is created by interception, so the cost is inseparable from the security benefit. Overhead is proportional to syscall rate: o = f_sys x 1.8 us.
- **Scaling.** Per-sandbox overhead does not depend on n directly, but I/O-heavy workloads multiply syscall rate and Sentry/Gofer threads, so host context switches and memory per sandbox (about 0.4 GB) grow linearly with n. Break-even against a 10% SLO is f_sys = 0.111 / 1.8 us = 61,700 syscalls/s per vCPU.

Task 3: Quantitative Performance Model

3.1 CPU: effective per-VM capacity

Let r be the vCPU:pCPU overcommit ratio and u the average utilisation of each vCPU. The offered load per physical CPU is rho = u r / gamma (gamma = 1 with SMT as specified, 0.8 when SMT is disabled; see Task 7). Capacity is measured in units of one dedicated native core.

Steal time (fraction of time a runnable vCPU waits for a pCPU), fluid proportional-share model:

$$sigma(rho) = max(0,\ 1 - 1/rho)$$

Virtualization overhead o_a is the extra execution time per unit of native work:

$$o_KVM(r) = [\ f0\ (1 + alpha\ (r - 1)) + f_io / b\ ]\ x\ t_exit\ +\ Lambda\ x\ dW$$

$$o_runc = f_sys\ x\ delta_sec + o_cgroup \qquad o_gVisor = f_sys\ x\ (t_gv - t_sys)$$

Effective per-vCPU capacity and per-VM capacity:

$$C_a(r,u) = (1 - sigma) / (1 + o_a) \qquad C_VM = v\ x\ C_a(r,u)$$

Worked example, KVM at r = 4. f_exit = 2000 x (1 + 0.25 x 3) + 20000/4 = 3500 + 5000 = 8500 exits/s; 8500 x 1.5 us = 1.275%. EPT term = 2x10^5 x 60 ns = 1.2%. So o = 2.475%. With u = 0.6, rho = 2.4, sigma = 1 - 1/2.4 = 0.583, so C = 0.417/1.02475 = 0.407, and C_VM = 4 x 0.407 = 1.63 native cores. **gVisor:** o = 2x10^5 x 1.8 us = 0.36, so even at r = 1 C = 1/1.36 = 0.735.

Overcommit r	A KVM o / C(u=1)	A KVM C(u=0.6)	B runc C(u=1)	B runc C(u=0.6)	C gVisor C(u=1)	C gVisor C(u=0.6)
1:1	2.25% / 0.978	0.978	0.985	0.985	0.735	0.735
2:1	2.32% / 0.489	0.814	0.493	0.821	0.368	0.613
4:1	2.48% / 0.244	0.407	0.246	0.411	0.184	0.306
8:1	2.77% / 0.122	0.203	0.123	0.205	0.092	0.153

Table 1: Per-vCPU capacity from the CPU model. At u = 1 capacity is about 1/r (steal dominates); the architecture-specific overhead only shifts the curve by 2 to 36%.

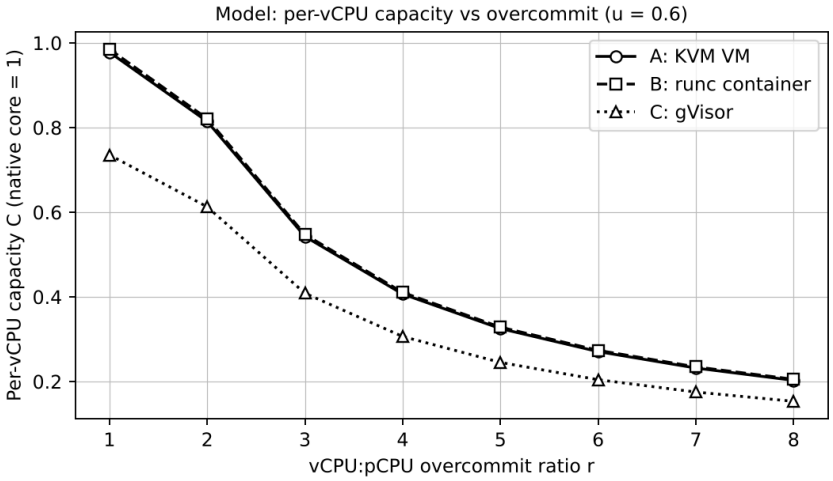


Figure 4: Model - per-vCPU capacity versus overcommit ratio at registration-peak utilisation.

Interpretation. Steal time (a scheduling effect) dominates once r exceeds about 2; the KVM and runc curves are within 1% of each other. Only gVisor is separated from them, by its syscall interception cost.

3.2 Memory: maximum VMs per host before ballooning, swapping or sharing

$$N_mem = floor((M - M_h) / (m_a + eps_a))$$

eps_a is per-instance overhead beyond the 8 GB demand. KVM: QEMU process 0.25 GB + EPT/page tables 0.2% of 8 GB = 0.27 GB. Containers have no guest kernel, so the 8 GB demand shrinks by about 0.5 GB (guest kernel and duplicate page cache) and eps = 0.05 GB. gVisor: same 0.5 GB saving, eps = 0.4 GB (Sentry, Gofer, double caching).

Results: KVM: (512 - 16)/(8 + 0.27) = 59.98, so **59 VMs**. runc: (512 - 12)/(7.5 + 0.05) = 66.2, so **66**. gVisor: 500/(7.5 + 0.4) = 63.3, so **63**. Memory overcommit extends this, with s = fraction shared by KSM and beta = fraction reclaimed by ballooning:

$$N_over = (M - M_h) / [(m_a + eps_a)(1 - s - beta)]$$

KVM: s = 0.15 gives 70 VMs; s = 0.15 and beta = 0.2 gives 92 VMs. runc: 77 and 101. gVisor: 74 and 97. Swapping begins when N x (m + eps)(1 - s - beta) exceeds M - M_h. Swap cost, with t_mem = 100 ns, t_swap = 100 us (NVMe) and fault probability p: EAT = (1 - p) t_mem + p t_swap. At p = 10^-3, EAT = 200 ns (2x slowdown); at p = 10^-2, EAT = 1.1 us (11x).

EduCloud check: 25 VMs x 8 GB = 200 GB, only 40% of host RAM, so memory is not the cause of the registration-week problem; CPU imbalance and the I/O path are (Task 4).

3.3 I/O and network: throughput versus number of sharing instances

Network. The aggregate ceiling is the smaller of the wire goodput and the CPU capacity of the packet path (k_net cores, 1500-byte packets), degraded by virtual-switch contention. With fair sharing among n active instances each with demand d:

$$T_cap,a(n) = min(eta\ B\ ,\ k_net\ x\ 12000\ bit\ /\ c_pkt,a\ )\ /\ (\ 1\ +\ kappa_a\ ln\ n\ )$$
$$T_inst(n) = min(\ d\ ,\ T_cap,a(n)\ /\ n\ )$$

Line goodput = 0.941 x 25 = 23.5 Gbps. CPU ceilings: KVM 4 x 12000/0.55 us = 87 Gbps; runc 64 Gbps; gVisor 16 Gbps (so gVisor is CPU-bound, not wire-bound). **Worked example (n = 8, KVM):** 23.5/(1 + 0.02 ln 8) = 22.6 Gbps aggregate, so 2.82 Gbps per VM.

n instances	A KVM: aggregate / per-inst	B runc: aggregate / per-inst	C gVisor: aggregate / per-inst
1	23.5 / 23.5 Gbps	23.5 / 23.5	16.0 / 16.0
4	22.9 / 5.73	22.6 / 5.65	15.0 / 3.74
8	22.6 / 2.82	22.2 / 2.77	14.5 / 1.81
16	22.3 / 1.39	21.7 / 1.36	14.1 / 0.88

Table 2: Network throughput predicted by the model (Gbps).

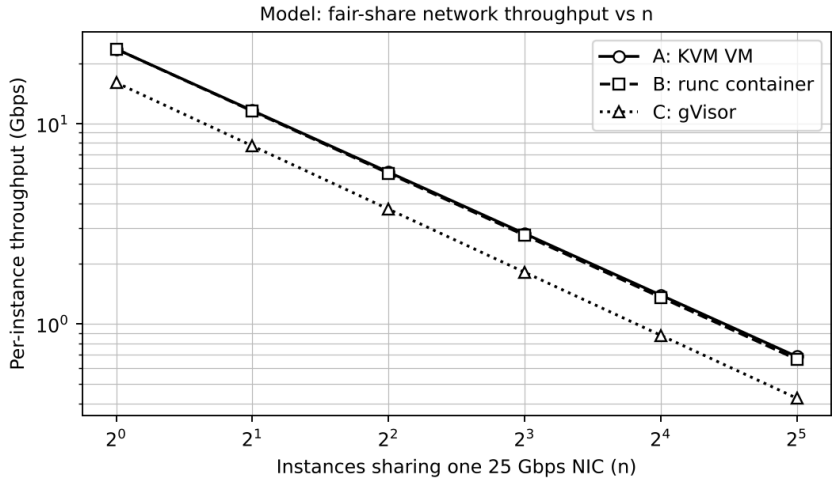


Figure 5: Model - per-instance network throughput versus number of instances sharing one 25 Gbps NIC.

Disk. IOPS ceiling: R_a = min(R_dev , k_io / c_io,a), and per-instance share R_a / n. KVM: min(500k, 8/4.4 us = 1.82M) = 500k, device-bound. runc: min(500k, 2.42M) = 500k. gVisor: min(500k, 8/19 us) = **421k**, CPU-bound. At n = 16 each instance gets 31.3k, 31.3k and 26.3k IOPS respectively. Queueing at the device (M/M/1 approximation, mu = 500k IOPS): mean I/O time W = 1/(mu(1 - rho_dev)) = 4 us at rho = 0.5, 20 us at 0.9, 40 us at 0.95, 200 us at 0.99.

3.4 Density under a service-level target

The SLO is: degradation relative to a bare-metal baseline at most delta (10% for SIS/LMS). Degradation is D = 1 - (1 - sigma)/(1 + o). Requiring D <= delta gives the maximum tolerable steal and the maximum load per pCPU:

$$sigma_max = 1 - (1 - delta)(1 + o) \qquad rho_max = 1\ /\ [\ (1 - delta)(1 + o)\ ]$$

$$N_cpu = floor(rho_max \times gamma \times P / (u \times v)) \qquad N_a = min(N_cpu , N_mem , N_net , N_io)$$

If (1 - delta)(1 + o) >= 1, i.e. o >= delta/(1 - delta) = 11.1%, the SLO cannot be met even with zero steal and the density is 0.

Worked example (KVM, u = 0.6, v = 4): rho_max = 1/(0.9 x 1.02475) = 1.084; N_cpu = floor(1.084 x 64/2.4) = floor(28.9) = **28**. N_mem = 59, so density = 28. **runc:** rho_max = 1/(0.9 x 1.015) = 1.095, N_cpu = 29. **gVisor:** o = 36% > 11.1%, so density = **0** for syscall-intensive SIS workloads at a 10% SLO.

Approach	u = 0.6 (peak), SMT on	u = 0.6, SMT off (gamma 0.8)	u = 0.2 (normal), SMT on	Limiting resource
A KVM	28	23	59 (CPU would allow 86)	CPU at peak, memory at normal load
B runc	29	23	66 (CPU would allow 87)	CPU at peak, memory at normal load
C gVisor	0 (o = 36%)	0	0	Syscall overhead; feasible only if f_sys < 61.7k/s

Table 3: Instances per host meeting the 10% SLO (4 vCPU, 8 GB instances).

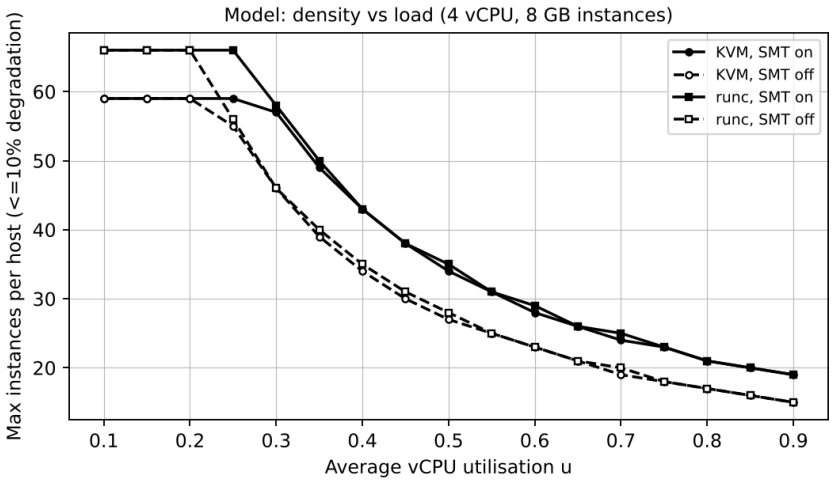


Figure 6: Model - maximum density versus average utilisation, with and without SMT.

Consequence for EduCloud. The present average of 25 VMs/host gives rho = 25 x 4 x 0.6/64 = 0.94, inside the SLO limit (1.084), so the average host is adequate. The reported failures therefore come from hot hosts (uneven placement, e.g. 36 SIS/LMS VMs on one host gives rho = 1.35, sigma = 26%) and from the shared storage and virtual network path, not from the mean load. A second key result: at peak the density difference between KVM (28) and containers (29) is a single VM per host (3.5%), so the choice between them should be decided by isolation, not by speed.

Task 4: Performance Under Different Operating Conditions

4.1 Best case

Conditions: CPU-bound or cache-friendly workload, vCPUs pinned to dedicated cores (r = 1:1), huge pages, no co-located noisy neighbours, large I/O batches.

A KVM. Guest code runs natively and exits only for timers. With 2 MB pages Lambda falls about 20x (the TLB covers more memory, so fewer nested walks), the EPT term drops from 1.2% to 0.06%; with no I/O, o = 2000 x 1.5 us + 0.06% = 0.36%. Latency and throughput are within 0.5% of bare metal; p95 latency is unchanged because a pinned vCPU is never descheduled (sigma = 0).

B runc. The kernel path is the native path; overhead is 0.5-1.5% (cgroup accounting plus seccomp). Best start-up (below 1 s) and best density because no guest OS.

C gVisor. For compute loops with few syscalls (say 10^3/s) o = 10^3 x 1.8 us = 0.18%, practically free: the sandbox is only expensive when the workload talks to the kernel. Best case is therefore a CPU-bound research job, not the SIS.

4.2 Typical case: registration-week mixed web and database load

Average per-host load rho = 0.94, but placement is uneven; assume a hot host at rho = 1.2. The mix is many short requests with syscalls and virtio events at a high rate.

A KVM. At rho = 0.94 steal is about zero and D = 2.3% (exits plus EPT), so mean latency rises only by the exit cost per I/O: for a 5-I/O login request roughly 5 x 1.5 us = 7.5 us on a 20 ms request, negligible. On the hot host, sigma = 1 - 1/1.2 = 16.7% and D = 1 - 0.833/1.0225 = 18.5%, breaching the SLO; the effect appears as queueing delay when a vCPU waits for a time slice, so p95 latency is hit harder than mean throughput. Inside the VM the guest also sees steal in /proc/stat, which is the monitoring signal.

B runc. Similar CPU behaviour (D = 1.5% at rho < 1, 17.9% on the hot host). Each login connection traverses veth, bridge and conntrack; with 29 containers sharing one bridge the model gives network efficiency g = 1/(1 + 0.03 ln 29) = 0.91, so about 9% of aggregate throughput is lost to switch contention, but the 23.5 Gbps line is far above login traffic.

C gVisor. D = 26.5% at rho < 1 because every connection setup, file read and database fsync is a trap: a request making 200 syscalls pays an extra 200 x 1.8 us = 0.36 ms, which on a 5 ms in-memory request is a 7% latency increase and a 26% CPU increase. This is why gVisor cannot host the SIS database tier.

4.3 Worst case

(a) Noisy neighbour (CPU). One tenant runs a 16-vCPU job at u = 1 next to 24 SIS VMs (4 vCPU, u = 0.6). Offered load: 24 x 4 x 0.6 + 16 = 73.6 pCPUs on 64, so rho = 1.15. If shares are not weighted (proportional model) sigma = 13%, giving D = 14.9% for victims, a violation. With equal per-vCPU weights, the scheduler is max-min fair: the water level lambda solves 96 x min(0.6, lambda) + 16 x min(1, lambda) = 64, so lambda = 0.571 and victim steal is 1 - 0.571/0.6 = 4.8%; mean throughput is protected but a victim vCPU must

still wait for its slice, so p95 latency rises by up to a scheduling quantum (several ms), and shared last-level cache and memory bandwidth (not in the CPU model) add further loss. For KVM the weight is given by libvirt shares/cpu.shares and can be capped; containers rely on cpu.weight/cpu.max, which throttles the aggressor in discrete periods (typical 100 ms) and can itself create latency spikes; gVisor adds Sentry threads that compete as additional runnable tasks.

(b) Memory overcommit with swapping. If hosts are packed to 92 VMs through ballooning and swap probability reaches p = 10^-2, EAT = 1.1 us versus 0.1 us: an 11x slowdown of memory access, which collapses database throughput (thrashing). KVM can swap guest pages invisibly to the guest (double paging), the worst case; a container exceeding memory.max is reclaimed and then OOM-killed, which is a failure but not a slowdown of neighbours; gVisor behaves like containers with larger footprints.

(c) Live migration during peak (KVM only; containers are restarted, not live-migrated). Pre-copy of an 8 GB VM over the 25 Gbps link at effective B_m = 2.8 GB/s with dirty rate R_d = 1 GB/s: ratio R_d/B_m = 0.357; rounds needed to bring the remainder below 0.1 GB: n = ceil(ln(0.1/8)/ln 0.357) = 5 rounds; total time T = V(1 - 0.357^6)/(B_m (1 - 0.357)) = 4.4 s; downtime = 8 x 0.357^5/2.8 = 17 ms. The hidden cost: dirty tracking write-protects pages, so each dirtied page raises an EPT violation: 1 GB/s / 4 KB = 250k faults/s x 1.5 us = 0.37 cores of CPU time burned on the VM, and the migration stream occupies about 22 Gbps, i.e. 95% of the 23.5 Gbps goodput of a shared NIC. Consequence: tenant network throughput collapses during migration unless the stream uses a dedicated NIC. If R_d approaches B_m the process does not converge.

(d) I/O storm. Sixteen VMs each issuing 100k IOPS (1.6M total) against a device of 500k IOPS: demand/capacity rho >> 1, each gets 31k IOPS (31% of demand), and queueing delay grows without bound (backlog); at rho_dev = 0.99 service time is 200 us versus 4 us at 0.5. KVM adds one exit per batch, so exit rate scales with IOPS (31k/4 = 8k exits/s per VM, 1.2% CPU); runc has no per-I/O trap and reaches the device ceiling; gVisor saturates at its own ceiling of 421k IOPS because Sentry and Gofer CPU saturate before the device does, so it degrades earlier but also shields the device slightly.

Condition	A KVM	B runc	C gVisor
Best case (CPU-bound, pinned)	D = 0.4%	D = 0.5-1.5%	D = 0.2% if syscalls are rare
Typical, rho = 0.94	D = 2.3%	D = 1.5%	D = 26.5%
Typical, hot host rho = 1.2	D = 18.5% (steal 16.7%)	D = 17.9%	D = 38% (0.833/1.36 = 0.61)
Noisy neighbour, rho = 1.15	Steal 4.8% (fair) to 13% (proportional); p95 latency up	Same CPU result; cgroup throttling jitter	Same plus Sentry thread contention
Swap, p = 10^-2	11x memory slowdown, double paging	OOM-kill instead of slowdown	OOM-kill
Live migration at peak	17 ms downtime, 0.37 core, 22 Gbps burst	Not supported (restart)	Limited (checkpoint/restore)
I/O storm, 16 x 100k IOPS	31k IOPS each, 1.2% CPU exit cost	31k IOPS each	26k IOPS each (CPU-bound)

Table 4: Summary of the model-derived operating-condition results, with the explanation in the text above (D = degradation relative to bare metal).

Task 5: Security Threat and Vulnerability Analysis

5.1 Attack surface per approach

Surface	A: KVM/QEMU	B: runc container	C: gVisor
Hypervisor / kernel	KVM module and CPU virtualization code; small, but a bug gives host control.	Entire host kernel (about 400 syscalls, all drivers, file systems); every kernel bug is reachable from every container.	Sentry (memory-safe Go) plus about 70 host syscalls; far smaller reachable kernel surface.
Device emulation	QEMU emulated devices and virtio backends (largest historical source of VM-escape CVEs). vhost moves part into the kernel.	None; devices are the host devices. Exposure instead through /proc, /sys, devices passed in.	Gofer and netstack replace kernel file/network code, with their own bugs.
Shared kernel	None between VMs (each has its own).	All containers share one kernel and its data structures.	Shared host kernel, but narrowly accessed.
Management plane	libvirt/OpenStack API, SSH, image store: root-equivalent access to every VM.	Container runtime socket (docker.sock), Kubernetes API, registries: root-equivalent.	Same as B plus runsc.
VM images and snapshots	qcow2 images and snapshots contain full disks and RAM state, and are copied freely (audit finding).	Layered images; smaller but pulled from registries; stale base layers.	Same as B.
Virtual network	Virtual switch/overlay; tenants on shared L2 can sniff/spoof without segmentation; migration traffic.	Shared bridge; CAP_NET_RAW by default allows ARP spoofing between containers.	Private netstack, so spoofing from the sandbox is limited to what the Sentry emits.

5.2 Threat categories, relative exposure and CIA impact on student records

Exposure scores run from 1 (low) to 5 (high). They are architectural judgements based on the trust boundaries in Task 1 (size of the enforcing component, how many tenants share it, and what an attacker gains). The weights w_k reflect impact on student records and are used again in Task 6.

Threat k	w_k	A KVM	B runc	C gVis.	Why the approaches differ	Impact (C / I / A) on student records
T1 Escape to host (VM escape, container breakout)	0.30	2	5	3	A needs a hypervisor/QEMU bug; B needs only a kernel bug reachable by any syscall; C must break the Sentry then seccomp.	C: all records on host exposed. I: modification. A: host crash takes all tenants.
T2 Cross-VM side channel (cache, SMT, speculative execution)	0.15	4	5	4	All share caches and SMT siblings; B additionally shares kernel page cache and kernel structures.	C: leakage of keys and tokens. I: none. A: none.

Threat k	w_k	A KVM	B runc	C gVis.	Why the approaches differ	Impact (C / I / A) on student records
T3 Hypervisor or runtime vulnerability	0.10	3	4	2	QEMU device model has a long CVE history; runc/kernel CVEs are frequent; Sentry smaller.	C/I/A depending on the bug.
T4 VM sprawl and unpatched guests	0.15	5	4	4	Each VM has its own kernel to patch (audit found outdated kernels); containers inherit host kernel patches but ship stale images.	I: compromise of old guest. A: ransomware, downtime.
T5 Image tampering / uncontrolled image copies	0.10	4	3	3	Multi-GB mutable qcow2 copies leave data at rest copies; registry layers can be signed easily.	C: student data in copied images. I: backdoored images.
T6 Insecure live migration	0.10	4	1	1	Only VMs migrate live; RAM is streamed in clear unless encrypted.	C: memory (passwords, records) sniffed. I: tampered state.
T7 Virtual-network sniffing / spoofing	0.10	3	4	2	Shared bridge and default capabilities favour B; gVisor has own stack.	C: interception of logins. I: spoofed requests. A: ARP/DoS.
Weighted exposure E_a = sum(w_k e_ak)/5	1.00	0.67	0.81	0.58	Computed: A = (0.6+0.6+0.3+0.75+0.4+0.4+0.3)/5; B = (1.5+0.75+0.4+0.6+0.3+0.1+0.4)/5; C = (0.9+0.6+0.2+0.6+0.3+0.1+0.2)/5.	

Table 5: Threat model and weighted exposure (lower is better).

Reading the result. gVisor has the smallest exposure index (0.58), KVM is intermediate (0.67) and runc the largest (0.81), although KVM is worse than gVisor on sprawl, image copies and migration, which are operational rather than architectural. Two facts matter for EduCloud: (i) the audit findings (outdated guest kernels, free image copying) map onto T4 and T5, which are exactly where KVM is exposed, so they must be fixed by process and mitigation (Task 7), not by changing approach; (ii) T1 and T2 are the threats that a regulator cares about when external tenants arrive, and only host-level separation removes T2 completely (an attacker who is not on the same host cannot run a cache attack). The exposure index is a ranking tool; sensitivity: halving the T1 weight from 0.30 to 0.15 and giving the 0.15 equally to T4 and T5 gives E = 0.75 (A), 0.77 (B) and 0.60 (C): runc stays worst, gVisor stays best, and KVM moves closer to runc. The qualitative ranking is therefore robust.

Task 6: Isolation-Performance Trade-off Formulation

The formulation uses the quantities derived in Task 3 (overhead o_a, steal sigma, N_mem, T_cap, rho_max) and Task 5 (exposure E_a, weights w_k).

Decision variables

- a_w in {KVM, runc, gVisor}: approach for each workload class w in {SIS, LMS, EXAM, RES (internal research), EXT (external big data)}.
- x_wh in {0,1}: class w placed on host h; n_wh: number of instances of w on host h; z_h in {0,1}: host h in use.

- gamma_h in {1, 0.8}: SMT on or off; r_h: overcommit ratio; m_h: set of mitigations on host h (KPTI, memory encryption, micro-segmentation, encrypted migration).

Objective

$$\text{minimise } J = \sum_h c_h z_h + \lambda \sum_w V_w \sum_h \sum_{t \neq w} x_{wh} x_{th} \tau_{wt} E(a_w, \gamma_h, m_h) + \mu \sum_w \max(0, D_w - \delta_w)$$

- **Cost term**: c_h is the annual cost of one host (hardware, power, licences), so the first term is the host count.
- **Risk term**: V_w is the value of the data in class w (SIS highest), tau_wt is the trust distance between two classes (0 for same owner, 1 for external versus student records), and the exposure of the pair is E(a, gamma, m) = Sum_k w_k e_ak (1 - rho_{m,k}) / 5, where rho_{m,k} in [0,1] is the fraction of threat k removed by the mitigations in m (for example SMT off removes about 0.6 of T2). Only co-resident pairs (x_wh x_th = 1) contribute, so placing classes on separate hosts removes the term entirely.
- **Performance term**: D_w = 1 - (1 - sigma)/(1 + o_a + o_m), where o_m is the extra overhead of mitigations; mu penalises SLO violation (soft constraint form).

Constraints

No.	Constraint	Origin
C1	D_w <= delta_w : 0.10 for SIS, LMS, EXAM; 0.20 for RES and EXT. Equivalent to sum of n_wh v_w u_w / (gamma_h P) <= rho_max(a_w, delta_w) = 1/((1 - delta_w)(1 + o_a + o_m)).	Task 3.4
C2	n_wh (m_w + eps_a) <= (M - M_h)(1 + s + beta) at the chosen overcommit; no swapping: s and beta limited to those in Task 3.2.	Task 3.2
C3	Sum_w n_wh g_w <= T_cap,a(n) where g_w is the per-instance network guarantee (about 0.2 Gbps for SIS; 2.5 Gbps for Spark workers).	Task 3.3
C4 (isolation)	x_SIS,h + x_EXT,h <= 1 for every host h (student records never co-resident with external tenants); a_SIS must be KVM or stronger, i.e. E(a_SIS) <= E_max.	Regulation, Task 5
C5	Sum_h z_h <= H (16 existing, plus any procured); Sum_h n_wh = N_w for every class (all instances placed).	Fleet
C6	gamma_h = 0.8 (SMT off) on any host h where two mutually untrusted tenants may run: x_EXT-A,h + x_EXT-B,h = 2.	Task 5, T2

How it combines. For a given placement the model chooses the cheapest set of approaches, hosts and mitigations that satisfy C1-C6; the risk term quantifies why isolation costs hosts: each constraint of type C4 and C6 reduces rho_max or gamma and therefore density N_a (Task 3.4), which raises the number of hosts sum z_h and hence cost. The optimum is found by enumeration (the problem is small: five classes, three approaches, a few mitigation sets) and is applied in Tasks 7, 11 and 12.

Task 7: Evaluation of Mitigation Strategies

Baseline for comparison (KVM, v = 4, u = 0.6, delta = 10%): o = 2.475%, density N = 28 per host, host fleet for 200 critical VMs = ceil(200/28) = 8 hosts. Each mitigation changes o, gamma or the host count, and the model is re-evaluated.

Cost computations

- **SMT off** (gamma = 1.25 to 1, i.e. the usable capacity falls to 0.8): N_cpu = floor(1.084 x 51.2/2.4) = floor(23.1) = 23. Density falls from 28 to 23, a loss of 17.9%. For 200 VMs hosts needed = ceil(200/23) = 9 (+1 host).
- **Kernel page-table isolation (KPTI / guest KPTI).** An extra CR3 switch per kernel entry, with PCID, about 0.1 us: o_kpti = f_sys x 0.1 us = 2%. KVM total o = 4.475%, rho_max = 1/(0.9 x 1.04475) = 1.0635, N = floor(1.0635 x 64/2.4) = 28. runc: o = 3.5%, N = floor(1.0735 x 64/2.4) = 28. Density cost is below 1 VM per host (rounding). Needed only on CPUs affected by Meltdown-class bugs.
- **Dedicated hosts for the critical pool.** Placing 200 critical VMs (u = 0.6) apart from 200 internal research VMs (u = 0.5): critical needs ceil(200/28) = 8, research needs ceil(200/floor(1.084 x 64/(0.5 x 4))) = ceil(200/34) = 6, total 14. A fully mixed pool needs total load/(per-host capacity): (200 x 4 x 0.6 + 200 x 4 x 0.5)/(64 x 1.084) = 880/69.4 = 12.7, so 13 hosts. Cost of dedication = +1 host (7.7%), which is small compared with the removal of T2 and T7 co-residency for SIS.
- **Hardware memory encryption (AMD SEV-SNP / Intel TDX), if the CPUs support it.** Memory-encryption bandwidth about 2%, nested-walk +20 ns per miss = 2x10^5 x 20 ns = 0.4%, and bounce-buffer copy raises c_pkt for KVM from 0.55 to about 0.95 us. o = 2.475% + 0.4% + 2% = 4.9%, rho_max = 1/(0.9 x 1.049) = 1.059, N = floor(1.059 x 64/2.4) = 28 (unchanged). CPU spent on packets at 23.5 Gbps rises from 1.96 Mpps x 0.55 us = 1.08 cores to 1.86 cores. Aggregate throughput unchanged (still wire-limited).
- **Image signing and verified boot.** No run-time overhead; adds a signature check at start (about 0.2 s per image) and an operational pipeline.
- **Micro-segmentation (per-VM firewall rules at the vSwitch).** About +0.1 us per packet, so c_pkt = 0.65 us; at line rate 1.96 Mpps x 0.1 us = 0.2 extra cores. Throughput stays wire-limited (87 Gbps CPU ceiling falls to 74 Gbps, above 23.5). Density unchanged.
- **Encrypted live migration (TLS/IPsec with AES-NI, 4 multi-fd streams).** About 4 GB/s per core of AES-GCM: 2.8 GB/s needs 0.7 core for 4.4 s, which does not change duration (the link is the limit). Extra cost is small and short.
- **Sandboxed runtime (gVisor) for any container exposed to untrusted code.** o = 36%, density factor 1/(1 + o) = 0.735, i.e. 26% fewer instances for syscall-heavy workloads.

Evaluation table

Mitigation	Threat addressed	Residual risk	Cost from Task 3 model	Acceptable for SIS?
SMT off (mixed or external-facing pools only)	T2 intra-core side channels	Cross-core cache (LLC) channels remain	Density 28 -> 23 (-17.9%), +1 host per 200 VMs	Only where untrusted co-residency exists. Not needed in the trusted SIS pool when pools are separated.
KPTI / guest kernel hardening	T2 (Meltdown class), kernel info leaks	Spectre variants, other microarchitectural leaks	+2% CPU; density 28 -> 28	Yes, where the CPU is affected.

Mitigation	Threat addressed	Residual risk	Cost from Task 3 model	Acceptable for SIS?
Dedicated hosts for SIS (host pools)	T1, T2, T7 across classes (removes co-residency)	Same-class tenants (SIS vs LMS) still share; host-level bugs	+1 host of 14 (7.7%)	Yes. Strongest benefit per unit cost.
Hardware memory encryption (SEV-SNP/TDX)	T1 consequences, T5 memory/disk at rest, malicious admin, T6	Side channels not removed; needs supporting CPUs and firmware trust	o 4.9%; N = 28; +0.8 core of packet CPU at line rate	Yes if supported; good for regulation evidence.
Image signing and registry control	T4, T5	Compromised signing key; signed but vulnerable image	No run-time cost; +0.2 s start	Yes, essential. Directly addresses the audit finding.
Micro-segmentation	T7	vSwitch or hypervisor compromise	+0.2 core at line rate; density unchanged	Yes.
Encrypted migration plus dedicated migration NIC	T6	Compromise of source or destination host	+0.7 core for 4.4 s; no throughput impact if NIC2 is dedicated	Yes.
Automated patching of guests (agents, image rebuild)	T4	Zero-day window	Operational (live migration during patching costs 0.37 core and a 17 ms stop per VM)	Yes.
gVisor for untrusted container code	T1, T3 for containers	Sentry bugs, netstack	Density x 0.735; I/O cap 421k IOPS	Not for SIS (breaks the 10% SLO); suitable for low-I/O student code.

Table 6: Mitigations with residual risk and model-based cost.

Conclusion for the SIS. The cheap mitigations (KPTI where needed, image signing, micro-segmentation, encrypted migration) cost nothing in density at the SLO because overcommit rounding hides their 2-5% CPU cost; the expensive ones (SMT off, gVisor) cost 18-26% of density. The best use of the budget is therefore host-pool separation plus the cheap mitigations, keeping SMT on in a pool where all tenants are trusted.

Task 8: Implement and Test

This section is the experiment plan and reproducible environment. Complete scripts are in Appendix A. The same resource allocations, workloads and repetitions are used for every approach so that results can be compared to Task 3.

8.1 Test bed

Item	Specification
Host	One Linux machine (Ubuntu 24.04, kernel 6.x) with VT-x/AMD-V enabled; record CPU model, core count, RAM and whether KVM is native or nested (`systemd-detect-virt`, `lscpu`). A laptop with nested virtualization is acceptable if documented, because nested VM exits cost 5-10x more (see Task 10).
Approach A	KVM/QEMU with libvirt or plain qemu-system-x86_64, -enable-kvm -cpu host, virtio-blk, virtio-net (vhost), Ubuntu cloud image, 2 vCPU and 2 GB per VM (scaled-down from 4 vCPU/8 GB).
Approach B	Docker with runc: --cpus=2 --memory=2g, default seccomp profile, default bridge network.
Approach C	Docker with the runsc runtime (gVisor): the same limits as B.
Baseline	Bare metal: same workloads run directly on the host with taskset to the same cores.
Scale	N = 1, 2, 4, 8, 16 co-located instances. If the machine has fewer cores, scale proportionally (e.g. keep vCPU:pCPU = 1:1, 2:1, 4:1, 8:1 by choosing instance size) and state the limitation.
Repetitions	5 runs per cell, 60 s each after a 10 s warm-up; report mean and standard deviation; drop caches between runs.
Mitigation toggles	SMT: `echo off > /sys/devices/system/cpu/smt/control`. KPTI: kernel parameter pti=on / nopti (where supported). Compare each with and without.

8.2 Test cases and their link to the model

Test	Tool and command (per instance)	Model quantity validated
CPU-bound	sysbench cpu --threads=2 --time=60 run; stress-ng --cpu 2 --timeout 60s --metrics	C_a(r,u): events/s versus N, steal from /proc/stat
Memory-intensive	sysbench memory --memory-block-size=1M --memory-total-size=50G run; stress-ng --vm 1 --vm-bytes 1G	EPT cost (Lambda x dW), N_mem, bandwidth
Disk I/O	fio --name=r --rw=randread --bs=4k --iodepth=32 --ioengine=libaio --direct=1 --runtime=60 --size=1G --output-format=json	R_a, IOPS per instance, p95 completion latency
Network	iperf3 -s on host; iperf3 -c HOST -t 60 -P 4 from each instance; and -u for latency/packet-rate	T_cap,a(n), g(n), c_pkt
Syscall micro-benchmark	Small loop calling getppid() 10^7 times; time per call	t_sys = 0.2 us, t_gv = 2.0 us
Interference (noisy neighbour)	Victim: sysbench cpu + fio + a latency probe (e.g. sockperf or wrk against nginx). Aggressor: stress-ng --cpu 4 --vm 2 --vm-bytes 90% --io 4 in a different instance, started after 30 s.	Steal under rho > 1 (max-min share), p95 latency
Start-up and footprint	Time from create to first response (`time docker run --rm img true`; boot to ssh for VM); RSS from `ps`, `docker stats`	eps_a, start-up
Security validation (see 8.3)	Commands below	T1, T2, T4 evidence

8.3 Security validation (to be demonstrated in the lab)

- **Kernel sharing:** run `uname -r` on the host and in each instance. Expected: identical in runc (shared kernel, trust boundary = kernel); different in the KVM guest (own kernel); in gVisor `dmesg` shows the Sentry synthetic kernel messages, so the guest does not see the host kernel log.
- **Namespace and cgroup isolation:** in a runc container `ls -l /proc/self/ns` and `lsns` show different namespace IDs from the host; `cat /sys/fs/cgroup/.../memory.max` shows the limit; prove enforcement by allocating more than memory.max and observing the OOM kill.
- **Mitigation status:** `grep . /sys/devices/system/cpu/vulnerabilities/*` and `cat /sys/devices/system/cpu/smt/active` before and after enabling a mitigation; record the status string (e.g. "Mitigation: PTE Inversion; VMX: ...").
- **Capabilities:** `capsh --print` inside a container shows the default capability set (including CAP_NET_RAW), evidence for T7.

Appendix A contains the scripts: environment setup (A.1), orchestration for N co-located instances (A.2), the workload runners (A.3), the interference test (A.4), the security checks (A.5) and the plotting script (A.6).

Task 9: Record Experimental Performance

Measured cells are blank on purpose. Fill them with the mean +/- standard deviation of 5 repetitions from the scripts in Appendix A. Predicted columns are computed from the Task 3 model so that Task 10 can compare directly.

9.1 Metrics to record

- Throughput (sysbench events/s, fio IOPS and MB/s, iperf3 Gbps), latency mean and 95th percentile (sysbench --percentile=95, fio clat p95, wrk/sockperf p95).
- CPU steal time (%, from /proc/stat inside VMs; `mpstat -P ALL 1` on the host), context switches (`perf stat -e context-switches -a`, `pidstat -w`), VM exits (`perf stat -e kvm:kvm_exit -a` or `kvm_stat -1`) for Approach A.
- Start-up time and memory footprint per instance; degradation = 1 - (throughput / bare-metal throughput).
- With and without mitigation: SMT on/off, KPTI on/off (and optionally memory encryption if supported).

9.2 Table 7: CPU-bound scaling, predicted versus measured

Prediction basis: a host with P = 8 pCPUs and 2-vCPU instances, u = 1 (sysbench), CPU-bound with few syscalls (o_KVM = 0.36%, o_runc = 0.5%, o_gVisor = 0.18%). Load rho = 2N/8; sigma = max(0, 1 - 1/rho); predicted per-instance throughput relative to one bare-metal instance C = (1 - sigma)/(1 + o). Rescale P and v to your machine.

N	rho	steal (model)	A KVM: pred / measured	B runc: pred / measured	C gVisor: pred / measured
1	0.25	0	0.996 / ____ +/- ____	0.995 / ____ +/- ____	0.998 / ____ +/- ____

N	rho	steal (model)	A KVM: pred / measured	B runc: pred / measured	C gVisor: pred / measured
2	0.5	0	0.996 / ____ +/- ____	0.995 / ____ +/- ____	0.998 / ____ +/- ____
4	1.0	0	0.996 / ____ +/- ____	0.995 / ____ +/- ____	0.998 / ____ +/- ____
8	2.0	0.50	0.498 / ____ +/- ____	0.498 / ____ +/- ____	0.499 / ____ +/- ____
16	4.0	0.75	0.249 / ____ +/- ____	0.249 / ____ +/- ____	0.249 / ____ +/- ____

9.3 Table 8: Summary template (one row per approach, N = 8 and N = 1)

Metric	Bare metal	A KVM	B runc	C gVisor	Model prediction (A / B / C)
fio 4 KiB randread IOPS, per instance, N = 8	____	____ +/- ____	____ +/- ____	____ +/- ____	62k / 62k / 52k (device 500k / 8; scale to your NVMe)
fio p95 latency (us)	____	____	____	____	A: +1.4 us exit cost per batch; C: +10 us Gofer RPC
iperf3 aggregate (Gbps), N = 8, 25G or loopback-limited	____	____	____	____	22.6 / 22.2 / 14.5 (scale to your NIC)
sysbench memory MB/s, N = 1	____	____	____	____	A within 1-2% of baseline (EPT); B, C about equal
getppid() latency (us)	0.2 expected	____ (guest, no exit)	____	____	0.2 / 0.2+0.05 / 2.0
CPU steal at N = 8 (%)	n/a	____	n/a (use cpu.stat throttled)	____	50% (rho = 2) in the CPU test
VM exits per second per vCPU (kvm_stat)	n/a	____	n/a	n/a	(2000 + 5000) at r = 1 under the mixed load
Context switches per second (host)	____	____	____	____	Highest for C (Sentry threads)
Start-up time (s)	n/a	____	____	____	15-40 / 0.3-1 / 0.6-2 (reasoned ranges)
Memory per instance (MB RSS over workload)	n/a	____	____	____	eps: 270 / 50 / 400 per 8 GB-class instance
Degradation vs bare metal (%)	0	____	____	____	Task 3: 2.3 / 1.5 / 26.5 (mixed web load)
With mitigation: SMT off, throughput change (%)	n/a	____	____	____	-20% capacity at rho >= 1
With mitigation: KPTI on, change (%)	n/a	____	____	____	-2% (syscall-heavy), about 0 CPU-bound

Metric	Bare metal	A KVM	B runc	C gVisor	Model prediction (A / B / C)
p95 latency, victim, noisy neighbour on (ms)	____	____	____	____	Steal 4.8% (fair share); p95 up by a scheduling quantum

9.4 Required graphs

Run `python3 plot_results.py results.csv` (Appendix A.6). It produces **Figure 7: per-instance throughput versus number of instances** (one line per approach plus the model prediction as a dashed line, with standard-deviation error bars) and **Figure 8: p95 latency of a victim instance with and without the noisy neighbour** (grouped bars, error bars). Figures 4 and 5 (Task 3) already contain the model curves to overlay.

Task 10: Compare Theory with Experiment

This section gives the method and the hypotheses that the measurements will confirm or refute, with the expected size and direction of each model error. Replace the expected values with your measured ones.

10.1 Error analysis procedure

$$e_i = (Measured_i - Predicted_i) / Predicted_i \qquad MAPE = (1/n)\ Sum\ |e_i|$$

- Acceptance: |e| <= 15% for throughput, <= 25% for p95 latency (tail latency is harder to model with a fluid steal model). A trend is "in agreement" if the sign and ordering of degradation across N and across approaches match even when |e| is larger.
- Calibrate before judging: fit t_exit from the KVM CPU/IO test (t_exit,fit = (o_measured - Lambda dW) / f_exit measured by kvm_stat), fit t_gv from the getppid() loop, c_pkt,a from iperf3 (c_pkt = k_net x 12000 / T_cap), and then recompute the Task 3 tables with the fitted values. A model that agrees only after calibration shows which assumed constant was wrong.

10.2 Hypotheses and expected causes of deviation

Prediction (Task 3/4)	Expected measured behaviour	Plausible cause of the difference
Per-instance CPU throughput follows 1/rho once rho > 1 for all three approaches, differing by <3% for A and B.	Agreement within 5-10%; the knee at rho = 1 is smoother than the model.	The fluid model ignores scheduler quantum and wake-up latency; CFS gives partial service before rho = 1; SMT adds up to 25% extra throughput beyond the P = 64 assumption.
KVM overhead 2-3% (mixed load), 0.4% CPU-bound.	Higher on a laptop: 5-20%.	Nested virtualization: an L2 exit is handled by L0 then forwarded to L1, so t_exit rises 5-10x (7.5-15 us); fit t_exit and the model recovers. Also turbo and thermal throttling on laptops.
runc overhead 1.5%, equal to bare metal within error.	Agreement; seccomp cost may be lower.	Syscall filter cost depends on the number of rules and kernel version.

Prediction (Task 3/4)	Expected measured behaviour	Plausible cause of the difference
gVisor overhead = f_sys x 1.8 us (about 36% at 2x10^5 syscalls/s); iperf3 ceiling about 16 Gbps; fio cap about 421k IOPS.	Same ordering; magnitude differs by platform: ptrace much worse than KVM/systrap platform.	Platform choice (ptrace, KVM, systrap) changes t_gv from about 2 us to 20 us; Gofer cache and direct file access reduce I/O cost; page cache duplication.
Network per-instance throughput = T_cap/n with efficiency g(n).	Aggregate flat or slowly falling for A and B; lower for C.	Loopback or virtual NIC on a laptop may limit well below 25 Gbps; interrupt coalescing, GRO/TSO and cache effects not modelled; per-flow TCP window limits.
Disk IOPS shared fairly; A and B near native, C lower.	A within 5-10% of B; C clearly lower; large variance across repetitions.	Host page cache (use O_DIRECT), NVMe write-amplification, SSD thermal state, qcow2 metadata overhead for KVM.
Noisy neighbour: victim steal 4.8% under fair share, p95 latency up by up to one scheduling quantum.	Throughput loss smaller than proportional model (13%) but p95 degraded more than mean.	Shared LLC and memory bandwidth contention (not in the CPU model) and CFS wake-up preemption; cgroup cpu.max throttling creates periodic stalls in B.
Start-up: container < gVisor << VM; memory per instance: runc < KVM < gVisor.	Expected ordering holds; absolute VM boot depends on image.	Cloud-init and image size; use a minimal kernel to cut VM boot to a few seconds.

10.3 Scalability and effect on the security conclusions

Which scales better. The model predicts that for CPU-bound work A and B scale identically (both limited by 1/rho), for network-heavy work A and B are both wire-limited with 4 packet-processing cores (CPU ceilings 87 and 64 Gbps against 23.5 Gbps goodput), so they scale equally until the wire saturates, with B using more CPU per packet (0.75 us against 0.55 us); for syscall- or I/O-heavy work C degrades earliest. Start-up and memory per instance make B scale best in instance count, A in isolation per instance. Decide using the slope of per-instance throughput against N in Figure 7 and the steal percentage at N = 8 and 16.

Decision rules for the security conclusions (Task 5).

- If `uname -r` is identical in the container and host, T1 exposure for B is confirmed. If the KVM guest shows a different, older kernel, T4 for A is confirmed and strengthens the patching mitigation.
- If the vulnerabilities directory shows "Vulnerable" for a Spectre/MDS entry on the lab CPU, T2 is open and SMT-off or microcode updates are mandatory for mixed-tenant pools.
- If the noisy-neighbour experiment shows victim p95 latency worsening much more than the model predicts, the cost of co-residency is higher than assumed, which supports separating the SIS onto its own pool even without a security argument.
- If SMT-off costs less than 20% capacity on the lab CPU, reduce gamma in the density formula and recompute; the conclusion "keep SMT on only where all tenants are trusted" holds whenever the cost is above about 10%.

Task 11: Recommended Final Architecture (SIS and LMS)

11.1 Recommendation

A hybrid centred on hardware-assisted KVM virtual machines. The tenant boundary for SIS, LMS and the examination portal is a KVM VM on a dedicated host pool; inside a VM, trusted stateless services (LMS web/API tiers) may run as runc containers for density. gVisor is reserved for running untrusted code such as student programming submissions, where I/O is low. Hosts: Linux with KVM, virtio/vhost, SMT on in the trusted pool, KPTI/microcode as required by the CPU, 2 MB huge pages for database VMs, vCPU pinning for the SIS database VMs, dedicated migration/storage NIC.

11.2 Why it is better than the alternatives

Criterion	A KVM VM	B runc	C gVisor
Density at peak, 10% SLO (Task 3.4)	28 / host	29 / host (+3.5%)	0 (o = 36%)
Overhead, mixed load	2.3%	1.5%	26.5%
Weighted exposure E (Task 5)	0.67	0.81 (shared kernel)	0.58
Fits SIS database I/O path	Yes (SR-IOV or vhost)	Yes	No (421k IOPS cap, Gofer)
Live migration for patching	Yes	No (restart)	Limited
Regulatory evidence of isolation	Strong (hardware boundary)	Weak	Medium

The model shows that at registration-peak load the difference between KVM and containers is one VM per host, 3.5%, while runc has the highest exposure and exposes student records to a kernel bug in any co-resident container. gVisor has the best exposure index but cannot meet the 10% SLO for the syscall-intensive SIS. So KVM is neither the fastest nor the most secure option; it is the point where both constraints are satisfied. The Task 6 function selects it because it is the only approach with C1 satisfied and E(a_SIS) below the isolation limit.

11.3 Modelled and observed performance

Model: o = 2.475% (r = 4, mixed load), N = 28 per host at u = 0.6, N = 59 at u = 0.2 (memory-bound), fleet of 8 hosts for 200 critical VMs, 14 hosts together with internal research. Observed: fill in from Table 7 and Table 8; the recommendation stands as long as measured KVM degradation is under about 10% at rho <= 1 and the measured KVM-versus-runc gap stays under about 10 percentage points. If nested virtualization inflates your KVM numbers, report the fitted t_exit and the recomputed density.

11.4 Security posture and residual risk

- Closed: T4/T5 (signed golden images, registry, no free copying, automated patching), T6 (encrypted migration on NIC2), T7 (micro-segmentation), T2 across classes (dedicated hosts).

- Residual: QEMU/KVM escape (T1) at low probability but high impact; cross-VM cache channels between SIS and LMS VMs on the same host; management-plane compromise. Reduced by sVirt/seccomp for QEMU, minimal device models, hardware memory encryption if supported, and strict RBAC with audit logging on the management plane.

11.5 Operational cost

Area	Operational consequence
Patching	One guest kernel per VM (the audit finding). Fix: rebuild from a patched golden image monthly and roll through live migration; hypervisor hosts patched with live migration; container images rebuilt in CI.
Image management	Signed images only, in a registry with versions and an SBOM; qcow2 copies are prohibited (use backing files with access control); encrypt snapshots at rest.
Migration	Encrypted, over NIC2; each migration costs about 0.4 core and 17 ms downtime per 8 GB VM (Task 4.3); avoid during registration week.
Monitoring	Steal %, exit rate, p95 latency of login and query, cgroup throttling, per-host rho against rho_max = 1.08, host kernel and CPU vulnerability status.

11.6 Limitations and growth from 400 to thousands

Limits: the model is fluid and ignores cache and memory-bandwidth interference; constants are assumed until calibrated; the threat scores are judgement-based. Growth: with 28 VMs per host at peak, 400 instances need 15 hosts, 2000 need 72 and 4000 need 143 hosts (at u = 0.6); at normal load (u = 0.2) memory limits to 59 per host. Practical issues at scale are then not the hypervisor but the control plane (OpenStack or Kubernetes with KubeVirt), the scheduler (keep rho at or below 1.0 per host, anti-affinity for SIS replicas), VM image sprawl and the shared storage back end. Containers inside VMs give the density of B when many small services appear, and the pool structure of Task 12 scales by adding hosts to a pool without changing the isolation argument.

Task 12: Day-4 Design Challenge - Hosting External Big-Data Tenants

A. Impact Analysis

Assumptions that no longer hold.

- **Trusted tenants:** all tenants were internal departments; external collaborators are less trusted, so T1 (escape), T2 (side channel) and T7 (network) now have a realistic attacker, and tau_wt = 1 between EXT and SIS.
- **Workload mix:** latency-sensitive, bursty, short requests (SIS/LMS) now share hardware with hours-long throughput jobs that run at u = 0.8 on all cores.
- **I/O profile:** the original model used 0.2 Gbps and about 20k virtio events per second per vCPU. Spark shuffle uses about 2.5 Gbps sustained per worker (assumed; bursts of 10 Gbps) and random/sequential disk at 100k IOPS, which drives the device to a utilisation where queueing, not virtualization overhead, dominates.
- **Maintenance:** live migration of running VMs competes with tenant traffic for the NIC (22 Gbps for 4.4 s per 8 GB VM) and adds 0.37 core of dirty tracking.

- **Peak load:** 3x registration load on SIS with the external jobs still running.

Effect on performance. On a mixed host of 20 SIS VMs (48 vCPU at u = 0.6) plus 4 Spark workers (8 vCPU, u = 0.8, so 25.6): rho = 73.6/64 = 1.15. Fair-share scheduling holds SIS steal at 4.8% (Task 4.3) and D = about 7%, which meets the 10% throughput SLO on average, but a proportional share would give sigma = 13% and D = 14.9%, and p95 latency suffers. If Spark drives the NVMe to rho_dev = 0.9 the mean I/O time is 20 us against 4 us at 0.5 (5x). Spark shuffle bursts of 4 x 10 Gbps exceed 23.5 Gbps goodput, so SIS packets queue behind them on the shared vSwitch.

Effect on the threat model. Threats rise from 0 to realistic likelihood for external tenants: an attacker can now rent a VM and run cache-timing code (T2), look for QEMU/kernel bugs (T1) and spoof network traffic (T7), while the target (student records) is hosted on the same hardware. Even a low-probability event has regulatory consequences, and the new regulation asks for demonstrable isolation, i.e. evidence that no external workload ever shares a host, kernel or network segment with SIS data.

B. Evaluation of the Original Design

- **Performance target.** The original design (KVM, 28 VMs per host at u = 0.6, delta = 10%) still meets the SLO for the SIS pool by itself (8 hosts). When the 48 external workers are added to the same pool, hosts are overloaded: using the Task 3 formula for the combined load (200 x 4 x 0.6 + 200 x 4 x 0.5 + 48 x 8 x 0.8 = 480 + 400 + 307 = 1187 vCPU-equivalents) divided by 64 x 1.084 = 69.4 gives 17.1, i.e. 18 hosts against 16 owned, so the SLO cannot be met without new capacity even before isolation is considered.
- **Isolation.** Constraint C4 (no co-residency of SIS and EXT) is violated. KVM provides a strong boundary (E = 0.67), but the regulation requires evidence, and T2 channels exist between any two VMs on the same host. Therefore the original design is not adequate for external tenants.
- **Alternative architecture.** gVisor or Kata for the external tenants: gVisor has lower exposure (0.58), but o = 36% already exceeds the limit delta/(1 - delta) = 25% for a 20% SLO, so it is infeasible for syscall-heavy shuffle; relaxing the SLO to 30% gives rho_max = 1/(0.7 x 1.36) = 1.05, N_cpu = 8, but the netstack cap of 14.5 Gbps gives N_net = floor(14.5/2.5) = 5 workers per host, hence ceil(48/5) = 10 hosts instead of 6 (+4), plus an I/O cap of 421k IOPS, for a small marginal security gain once external tenants are on their own hosts. Kata (micro-VM) is a KVM configuration and would add little over a KVM VM with a minimal device model. Containers (B) for external tenants are rejected because of the shared kernel (E = 0.81).

C. Adaptation: MODIFY (not retain, not a full redesign)

Decision: modify. Keep KVM as the isolation technology everywhere (the model still supports it), but separate tenants by host pool and add network/storage separation and a tailored configuration for the big-data pool.

Pool	Tenants	Technology and settings	Hosts (model)
P1 Critical	SIS, LMS, EXAM (200 VMs, trusted, u = 0.6)	KVM VMs, SMT on (no untrusted co-tenants), KPTI where needed, SEV-SNP if available, vCPU pinning for DB VMs, encrypted live migration on NIC2, micro-segmentation.	ceil(200/28) = 8

Pool	Tenants	Technology and settings	Hosts (model)
P2 Internal research	Departmental workloads (200 VMs, u = 0.5)	KVM VMs or runc inside VMs, SMT on, priority class that is throttled in registration week.	ceil(200/34) = 6
P3 External big data	Collaborator Spark/Hadoop (48 workers, 8 vCPU, 32 GB, u = 0.8)	KVM VMs, one VM group per collaborator, SMT off, SR-IOV VF per worker with rate limit, local NVMe namespace or dedicated volume per tenant, no live migration (drain and reschedule), separate VRF/VLAN and firewall to P1/P2.	See below: 6

Sizing P3 with the Task 3 model. Per-host density for external workers with delta = 20%: with SR-IOV the exit load from network drops, so o = 1.5%, rho_max = 1/(0.8 x 1.015) = 1.23. With SMT off P_eff = 51.2: N_cpu = floor(1.23 x 51.2/(0.8 x 8)) = floor(9.8) = 9. Memory: (512 - 16)/(32 + 0.31) = 15. Network: 23.5 Gbps / 2.5 Gbps = 9.4, so 9. **N = min(9, 15, 9) = 9 workers per host**, so 48 workers need ceil(48/9) = 6 hosts. If the real shuffle averages 5 Gbps, N_net = 4 and 12 hosts are needed, so the NIC, not the CPU, is the design variable: sustained network per worker should be put in the contract.

Total hosts: 8 + 6 + 6 = 20 versus 16 owned, a shortfall of 4. Options: (i) purchase 4 hosts, charged to the collaborators; (ii) throttle internal research (P2) during registration week, which cuts its u from 0.5 to 0.25 and frees about 3 hosts; (iii) cap external jobs in registration week. Recommended: (i) plus (ii), and treat P3 as a metered service.

Effects of the change.

- **Density:** the cost of the regulation is the loss of cross-class multiplexing: 20 hosts versus about 18 for a fully shared fleet, +11%, and SMT-off in P3 lowers the CPU-limited density from 10 to 9 workers per host, which equals the network limit (9), so it costs nothing extra there.
- **Throughput:** SR-IOV gives the Spark workers the full NIC goodput (23.5 Gbps per host shared by 9 workers, 2.6 Gbps each) and removes vhost CPU cost; the storage/migration NIC (NIC2) is dedicated to P1/P2 migration and back-end traffic, so a 4.4 s migration burst no longer competes with tenants. A running migration is capped by auto-converge when R_d approaches the link rate.
- **Attack surface:** external tenants have no network path, kernel or hardware in common with student records; T2 is removed between P3 and P1 (different hosts), reduced within P3 by SMT-off; T6 is removed for P3 (no live migration); T7 reduced by VRF and micro-segmentation; management plane gets separate projects, RBAC and audit trail. The only shared element is the control plane and (optionally) the storage back end, which is encrypted per tenant.
- **Patching:** P1/P2 hosts by live migration outside registration week; P3 hosts by drain and reboot (Spark re-executes lost tasks), which eliminates the migration/SR-IOV conflict.

D. Evidence-Based Defence

- **Model.** Density formula (Task 3.4): KVM 28/host at peak and 9 external workers/host; the Task 6 cost function J has three feasible candidates: shared fleet (violates C4, infeasible), modified pools with KVM (20 hosts, E_pair = 0 across classes), and gVisor for P3 (infeasible at delta = 20%; with delta = 30% it needs 10 hosts for P3, 24

in total, for an exposure lower by 0.09). The modified design has the lowest J among feasible options.

- **Experiments.** The noisy-neighbour test (Task 8/9) validates the 4.8% steal and the p95 degradation that motivate separating batch from latency-sensitive classes; the mitigation tests show whether SMT-off (-20%) and KPTI (-2%) match the model, which decide where each is applied (SMT off only in P3). If the measured noisy-neighbour p95 degradation is smaller than predicted, pool separation is still required by C4, so the decision does not depend on this result; it only changes the number of hosts. If measured gVisor overhead is far below 36% (for example with a different platform), re-run the sizing: gVisor could then be considered as an additional layer for P3, but only with Spark I/O measured.
- **Changed workload and tenancy.** Big-data I/O (2.5 Gbps and 100k IOPS per worker) makes the shared NIC and NVMe the bottleneck (N_net = 9, rho_dev = 0.8 to 0.9); SR-IOV and a dedicated NIC for storage/migration address precisely those two resources.
- **Cost and density trade-off.** The price of demonstrable isolation is +4 hosts relative to 16 (and about +2 relative to an unisolated shared design); this is the explicit trade-off quantified by J. The alternative of keeping 16 hosts requires giving up either the 10% SLO for SIS or the regulation, neither of which is acceptable.
- **Conclusion.** MODIFY: retain KVM and the original mitigations, introduce host pools, SR-IOV and encrypted/dedicated migration, and add 4 hosts. This follows from three results: isolation cost (C4) removes the possibility of sharing, the I/O profile makes the NIC the limiting resource in P3, and the overhead model shows that stronger-but-slower sandboxes (gVisor) fail the SLO in the SIS pool and bring little extra security in an externally-dedicated pool.

Appendix A: Reproducible Scripts

All scripts run on Ubuntu 24.04. Paths and image names are examples; adjust to your machine. Run as a user in the docker, libvirt and kvm groups.

A.1 setup.sh - environment

```
#!/bin/bash
set -e
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system virtinst docker.io sysbench
fio iperf3 \
     stress-ng linux-tools-generic sysstat cloud-image-utils python3-matplotlib
python3-pandas
# gVisor (runsc): see https://gvisor.dev/docs/user_guide/install/ then:
sudo runsc install && sudo systemctl restart docker
# record the environment (needed for the report)
lscpu > env_lscpu.txt; uname -a > env_kernel.txt; systemd-detect-virt >
env_virt.txt
grep -E "vmx|svm" /proc/cpuinfo | head -1 > env_hwvirt.txt
grep . /sys/devices/system/cpu/vulnerabilities/* > env_vuln.txt
cat /sys/devices/system/cpu/smt/active > env_smt.txt
# Docker image with the benchmarks
cat > Dockerfile <<EOF
FROM ubuntu:24.04
RUN apt-get update && apt-get install -y sysbench fio iperf3 stress-ng util-
linux
COPY bench.sh /bench.sh
EOF
docker build -t edubench .
```

A.2 bench.sh - identical workload inside every instance (and on bare metal)

```
#!/bin/bash
# usage: bench.sh <tag> <iperf_server_ip>   writes CSV lines to stdout
TAG=$1; SRV=$2; T=60
steal() { awk '/^cpu /{print $9, $2+$3+$4+$5+$6+$7+$8+$9}' /proc/stat; }
read s0 t0 < <(steal)
# CPU-bound: events/s and 95th percentile latency
sysbench cpu --threads=2 --time=$T --percentile=95 run > /tmp/cpu.txt
echo "$TAG,cpu_events,$(awk '/events per second/{print $4}' /tmp/cpu.txt)"
echo "$TAG,cpu_p95_ms,$(awk '/95th percentile/{print $3}' /tmp/cpu.txt)"
read s1 t1 < <(steal)
echo "$TAG,steal_pct,$(echo "scale=3; 100*($s1-$s0)/($t1-$t0)" | bc)"
# memory-intensive
sysbench memory --memory-block-size=1M --memory-total-size=50G --threads=2 run
> /tmp/mem.txt
echo "$TAG,mem_MBps,$(awk '/transferred/{gsub(/[()]/,"");print $4}'
/tmp/mem.txt)"
# disk I/O
fio --name=r --rw=randread --bs=4k --iodepth=32 --ioengine=libaio --direct=1 \
    --size=1G --runtime=$T --time_based --output-format=json > /tmp/fio.json
python3 - <<PY
import json; d=json.load(open("/tmp/fio.json"))["jobs"][0]["read"]
print("$TAG,fio_iops,%s" % d["iops"]); print("$TAG,fio_p95_us,%s" %
(d["clat_ns"]["percentile"]["95.000000"]/1000))
PY
# network
iperf3 -c $SRV -t $T -P 4 -J | python3 -c "import
sys,json;d=json.load(sys.stdin);print('$TAG,iperf_Gbps,%.2f' %
(d['end']['sum_received']['bits_per_second']/1e9))"
# syscall micro-benchmark (0.2 us native expected)
python3 -c "import os,time;n=10**6;t=time.perf_counter();[os.getppid() for _ in
range(n)];print('$TAG,syscall_us,%.3f' % ((time.perf_counter()-t)/n*1e6))"
```

A.3 run_matrix.sh - N = 1, 2, 4, 8, 16, five repetitions, all approaches

```
#!/bin/bash
```

```
# prerequisite: iperf3 -s -D on the host; VM base image
/var/lib/libvirt/images/base.qcow2 with
# bench.sh, sysbench, fio, iperf3 installed (cloud-init) and an ssh key for
user "bench".
HOST_IP=$(hostname -I | cut -d" " -f1); mkdir -p results;
OUT=results/results.csv
echo "approach,N,rep,metric,value" > $OUT
for APP in baremetal kvm runc gvisor; do
 for N in 1 2 4 8 16; do
  for REP in 1 2 3 4 5; do
   sync; echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
   START=$(date +%s.%N)
   case $APP in
    baremetal) for i in $(seq $N); do (taskset -c $(( (i-1)*2 % $(nproc) )),$((
((i-1)*2+1) % $(nproc) )) \
                  ./bench.sh $APP $HOST_IP > results/tmp_$i.csv) & done ;;
    kvm) for i in $(seq $N); do
          qemu-img create -f qcow2 -b /var/lib/libvirt/images/base.qcow2 -F
qcow2 \
               /var/lib/libvirt/images/vm$i.qcow2 > /dev/null
          virt-install --name vm$i --memory 2048 --vcpus 2 --cpu host-
passthrough \
               --disk path=/var/lib/libvirt/images/vm$i.qcow2,bus=virtio --
import \
               --os-variant ubuntu24.04 --network network=default,model=virtio
\
               --noautoconsole; done
         # wait for boot: record time to first ssh success per VM as start-up
time
         for i in $(seq $N); do IP=$(virsh domifaddr vm$i | awk '/ipv4/{print
$4}' | cut -d/ -f1)
          until ssh -o StrictHostKeyChecking=no bench@$IP true 2>/dev/null; do
sleep 0.5; done
          echo "$APP,$N,$REP,startup_s,$(echo "$(date +%s.%N)-$START" | bc)"
>> $OUT
           (ssh bench@$IP "./bench.sh kvm $HOST_IP" > results/tmp_$i.csv) &
done ;;
    runc|gvisor) RT=runc; [ $APP = gvisor ] && RT=runsc
         for i in $(seq $N); do (docker run --rm --runtime=$RT --cpus=2 --
memory=2g edubench \
               bash /bench.sh $APP $HOST_IP > results/tmp_$i.csv) & done ;;
   esac
   # host-side counters while instances run (VM exits, context switches)
   sudo perf stat -a -e context-switches,kvm:kvm_exit -o
results/perf_${APP}_${N}_${REP}.txt sleep 60 &
   wait
   cat results/tmp_*.csv | awk -F, -v a=$APP -v n=$N -v r=$REP '{print
a","n","r","$2","$3}' >> $OUT
   rm -f results/tmp_*.csv
   [ $APP = kvm ] && for i in $(seq $N); do virsh destroy vm$i; virsh undefine
vm$i --remove-all-storage; done
  done
 done
done
```

A.4 noisy.sh - interference experiment (run for each approach with N = 8)

```
#!/bin/bash
# Instance 1 = victim (bench.sh), instance 2 = aggressor started after 30 s.
# Aggressor command, executed inside instance 2 (VM via ssh, container via
docker exec):
stress-ng --cpu 4 --vm 2 --vm-bytes 90% --io 4 --timeout 60s
# Compare victim p95 (cpu_p95_ms, fio_p95_us) and steal_pct against a run
without the aggressor.
# Cap the aggressor with cgroups / libvirt cputune to test the isolation
mechanism:
docker run --cpus=2 --cpu-shares=256 ...          # container
virsh schedinfo vm2 --set vcpu_quota=100000        # VM (100% of one pCPU)
```

A.5 security_checks.sh - security validation

```
#!/bin/bash
echo "== host kernel =="; uname -r
echo "== runc container kernel (expect SAME as host) =="; docker run --rm
ubuntu:24.04 uname -r
echo "== gVisor (expect gVisor kernel string, not the host) =="; docker run --
rm --runtime=runsc ubuntu:24.04 uname -r
echo "== KVM guest kernel (expect its OWN kernel) =="; ssh bench@$VMIP uname -r
echo "== namespaces: host vs container =="; ls -l /proc/self/ns; docker run --
rm ubuntu:24.04 ls -l /proc/self/ns
echo "== cgroup memory limit enforced =="; docker run --rm --memory=256m
ubuntu:24.04 \
     bash -c "cat /sys/fs/cgroup/memory.max; python3 -c
'x=bytearray(400*1024*1024)'" ; echo exit=$?
echo "== default container capabilities (CAP_NET_RAW) =="; docker run --rm
ubuntu:24.04 bash -c "apt-get -qq update;apt-get -qq install -y libcap2-
bin;capsh --print | grep Current"
echo "== CPU mitigations =="; grep . /sys/devices/system/cpu/vulnerabilities/*;
cat /sys/devices/system/cpu/smt/active
echo "== mitigation toggle =="; echo off | sudo tee
/sys/devices/system/cpu/smt/control; cat /sys/devices/system/cpu/smt/active
```

A.6 plot_results.py - Figures 7 and 8 (black and white)

```
import pandas as pd, matplotlib; matplotlib.use("Agg"); import
matplotlib.pyplot as plt
df = pd.read_csv("results/results.csv")
sty = {"baremetal":("k","x","-"),"kvm":("k","o","--
"),"runc":("k","s",":"),"gvisor":("k","^","-.")}
g =
df[df.metric=="cpu_events"].groupby(["approach","N"]).value.agg(["mean","std"])
.reset_index()
fig, ax = plt.subplots(figsize=(6.4,3.8))
for a, d in g.groupby("approach"):
    ax.errorbar(d.N, d["mean"], yerr=d["std"], color="k", marker=sty[a][1],
ls=sty[a][2], capsize=3, label=a, mfc="w")
ax.set_xscale("log", base=2); ax.set_xlabel("Co-located instances N");
ax.set_ylabel("sysbench events/s per instance")
ax.legend(); ax.grid(True, color="#bbb", lw=.5); fig.tight_layout();
fig.savefig("fig7_throughput_vs_N.png", dpi=170)
# Figure 8: p95 latency with / without noisy neighbour (add a "noisy" column to
the CSV, 0 or 1)
if "noisy" in df.columns:
    h =
df[df.metric=="cpu_p95_ms"].groupby(["approach","noisy"]).value.agg(["mean","st
d"]).unstack()
    h["mean"].plot.bar(yerr=h["std"], color=["w","0.5"], edgecolor="k",
capsize=3, figsize=(6.4,3.8))
    plt.ylabel("victim p95 latency (ms)"); plt.tight_layout();
plt.savefig("fig8_latency_noisy.png", dpi=170)
```